Discrimination of Objects Mounted on Optical Bench Using Ultrasonic Sensing and Machine Learning

Information Technology – Machine Learning

Rashid, Romana
Matriculation no: 1428733
romana.rashid@stud.fra-uas.de

Information Technology – Machine Learning

Prabhu, Mithila Maruti
Matriculation no: 1567111
mithila.prabhu@stud.fra-uas.de

***Abstract*—This paper presents an experimental study on the automated discrimination of objects mounted on an optical bench using a Red Pitaya ultrasonic sensing system combined with supervised machine learning classifiers. The objects under test represent a range of surface geometries and acoustic properties, including metallic, rigid plastic, foam-covered, and hollow plastic surfaces. Ultrasonic echo waveforms are acquired across a range of distances using the Red Pitaya platform, with multiple ADC measurements collected per acquisition to ensure signal stability. A multi-dimensional feature vector is extracted from each waveform, combining time-domain statistics such as mean amplitude, standard deviation, root mean square, and signal energy with spectral features derived from the Fast Fourier Transform. Three machine learning classifiers are trained on a dataset acquired on the first measurement day: Support Vector Machine, Random Forest, and K-Nearest Neighbours. The trained models are serialised with timestamps and subsequently reloaded to evaluate their performance on a fresh dataset acquired on a separate day without retraining. A secondary experiment trains and evaluates classifiers using measurements taken only at a small number of selected distances rather than the full measurement range. Results demonstrate that Random Forest achieves the highest in-session classification accuracy, which declines substantially when the model is applied to data from the second day, revealing significant temporal drift caused by environmental variation in the ultrasonic signal characteristics. When retrained on data from only the selected discrete distances, the Random Forest model recovers a notably higher cross-day accuracy, outperforming both SVM and KNN under all experimental conditions. The study confirms that ultrasonic classifiers are susceptible to inter-session performance degradation and identifies Random Forest combined with targeted distance-specific training as the most temporally stable configuration for this application.**

Keywords— ***Object discrimination, ultrasonic sensing, optical bench measurements, feature extraction, machine learning, Random Forest***

I. Introduction

Object discrimination using sensor signals is an important task in intelligent sensing and automated measurement systems. In applications such as robotics, industrial automation, and smart sensing environments, machines must be able to recognize and differentiate objects based on signals obtained from sensors. When an object interacts with a sensing device, the resulting signal contains information about its physical characteristics, including material properties, surface structure, and distance from the sensor. By analyzing these signals, objects can be automatically identified and classified without human intervention [1].

Recent advances in signal processing and machine learning have significantly improved the analysis of complex sensor data. Traditional signal processing techniques often rely on manually designed rules or threshold-based methods, which may not perform well when signals vary due to environmental conditions or object variability. Machine learning provides a more flexible approach by allowing algorithms to learn patterns directly from labeled datasets [2]. Once trained, these models can classify new signals based on the relationship between signal features and object classes.

Ultrasonic sensing is widely used for non-contact object detection and measurement. Ultrasonic sensors emit high-frequency acoustic waves that propagate through the surrounding medium. When these waves encounter an object, a portion of the signal is reflected back toward the sensor. The characteristics of the reflected signal depend on factors such as the material of the object, surface structure, orientation relative to the sensor, and the distance between the sensor and the object. These variations in reflected signals can therefore be used to discriminate between different objects [3].

In this project, ultrasonic signals were recorded from objects mounted on an optical bench using a Red Pitaya measurement platform. Red Pitaya is a compact open-source system that integrates high-resolution analog-to-digital converters and signal processing capabilities for acquiring and analyzing analog signals [4]. The optical bench provides a controlled experimental environment that allows precise positioning of objects relative to the ultrasonic sensor, ensuring consistent measurement conditions.

The recorded waveform signals were processed to extract meaningful information suitable for machine learning analysis. Statistical and frequency-domain features were obtained from each waveform to capture important signal characteristics while reducing data dimensionality. These features were used to construct a dataset for supervised

machine learning classification [5]. Several classification algorithms, including Random Forest, Support Vector Machine, and K-Nearest Neighbors, were trained to discriminate between the objects based on their ultrasonic signal responses.

The main objective of this study is to investigate the effectiveness of machine learning techniques for discriminating objects using ultrasonic sensor measurements obtained from an optical bench experiment. By combining signal processing methods with machine learning algorithms, this work aims to develop a reliable approach for identifying objects based on their ultrasonic signal responses and to demonstrate the potential of machine learning methods in intelligent sensing systems.

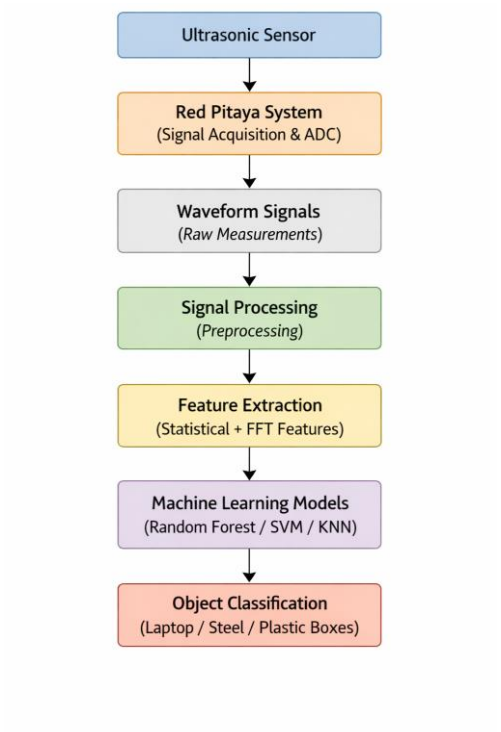


Fig 1: Components of Machine Learning

II. THEORITICAL BACKGROUND

A. RED PITAYA ULTRASONIC SYSTEM

Ultrasound (US) imaging is a common medical tool these days. It is feasible to construct an image of almost any organ or tissue in the human body by capturing the reflected signals of acoustic signal bursts at frequencies much above the human hearing limit, which are reflected by the various tissues with varying intensities. While this technology is undoubtedly comparable to other imaging methods like conventional radiography or CT scans, it also has the benefits of being less expensive and operating in a safer, radiation-free manner.

Red Pitaya is an open-source measurement and signal processing platform that integrates signal acquisition hardware, programmable logic, and software tools within a compact embedded system. The platform is widely used in research and engineering applications for capturing, processing, and analyzing analog signals. It provides high-resolution analog inputs, digital signal processing capabilities, and network connectivity, allowing real-time acquisition and analysis of sensor signals. In this project, Red Pitaya was used to acquire ultrasonic sensor signals and convert them into digital waveform data for further signal processing and machine learning analysis.

The hardware architecture of Red Pitaya includes analog input channels, an analog-to-digital converter (ADC), programmable logic, and an embedded processor. The ultrasonic sensor output is connected to the analog input channel of the Red Pitaya system. Since sensor signals are analog in nature, the analog-to-digital converter samples the incoming signal and converts it into a digital waveform. This digital representation allows the signal to be stored, processed, and analyzed using computational methods.

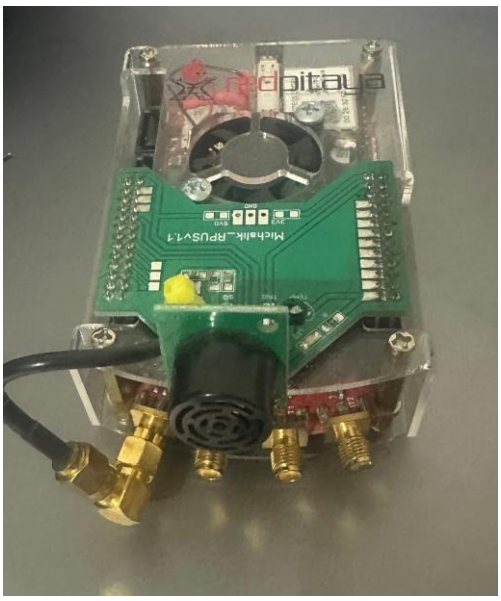


Fig 2: Red Pitaya measurement platform used for ultrasonic signal acquisition.

In this project, it is configured as an ultrasonic transceiver: the FPGA generates precisely timed transmit pulses to drive an ultrasonic transducer, while the received echo signal is digitized by a 14-bit analogue-to-digital converter (ADC) running at 125 Msps. The platform communicates with a host computer over a UDP network interface, and a dedicated GUI application (Frankfurt UAS, v0.23) controls acquisition parameters, displays the ADC and FFT waveforms in real time, and saves measurement files to disk. . This setup provides a stable and reproducible environment for acquiring ultrasonic measurements from objects mounted on an optical bench. [6]

The measurement software allows the user to configure the number of ADC measurements per acquisition. In all experiments reported here, a count of 50 measurements is used. Each measurement returns one ADC waveform (8192 samples) followed by one FFT waveform (8192 samples), giving a total of 100 waveform blocks per saved file. The data loader used in this project parses these files by splitting the continuous sample stream into alternating ADC and FFT blocks starting from sample index 300, which skips the file header.

B. Analogue-to-Digital Conversion (ADC)

The ADC is the hardware component responsible for converting the continuous-time analogue echo voltage at the ultrasonic receiver into a discrete-time digital sequence suitable for processing by the computer. The Red Pitaya ADC samples at 125 Msps with 14-bit resolution, providing a dynamic range of approximately 84 dB. Each captured waveform of 8192 samples represents approximately 65.5 microseconds of signal, sufficient to capture the primary echo from objects at distances up to approximately 11 metres in standard air condition.

For the short distances tested in this project (5 to 30 cm), the echo arrives well within the first few thousand samples of each waveform. The time-of-flight between the transmitted pulse and the peak of the echo envelope is directly proportional to the object distance, providing the primary geometric cue for distance estimation. However, the shape and amplitude of the echo also encode information about the surface geometry and material of the object, which is the basis for object discrimination. The resulting digital signal preserves important characteristics such as amplitude, timing, and waveform shape, which are essential for further signal processing and feature extraction. [7]

C. Fast Fourier Transform (FFT)

The Fast Fourier Transform (FFT) is an efficient algorithm for computing the Discrete Fourier Transform (DFT) of a discrete-time signal. It decomposes the time-domain waveform into its constituent frequency components, revealing the spectral content of the echo. For a real-valued signal of length N, the real FFT produces N/2 + 1 complex spectral coefficients, from which the magnitude spectrum is computed.

In the context of ultrasonic echo analysis, the FFT is valuable because different objects and surface geometries produce echoes with characteristic frequency distributions. Hard, flat surfaces (such as the steel plate) tend to produce spectrally compact echoes concentrated near the transducer resonance frequency, while complex or absorbing surfaces (such as foam-covered objects) diffuse energy across a broader spectral range. The FFT magnitude spectrum therefore complements the time-domain features in the classification feature vector.

In this project, five spectral features are extracted from the FFT magnitude of each ADC waveform: mean spectral amplitude, standard deviation of the spectrum, maximum spectral amplitude, the frequency bin index of the spectral peak, and the total spectral energy (sum of squared magnitudes). This transformation reveals the spectral content of the signal, including dominant frequencies and energy distribution. In ultrasonic sensing, different materials and surface structures produce characteristic frequency responses in the reflected signal [8].

D. Support Vector Machine (SVM)

A Support Vector Machine is a supervised learning algorithm that constructs a decision boundary in the feature space that maximizes the margin between classes. For linearly inseparable problems, the kernel trick maps input features into a higher-dimensional space where linear separation becomes feasible. In this project, the Radial Basis Function (RBF) kernel is used, defined as $K(x, x') = \exp(-\gamma\|x - x'\|^2)$, where γ controls the width of the kernel. The regularization parameter C controls the trade-off between margin maximization and training error. Both C and γ are set by cross-validation. SVMs are particularly well-suited to problems with a moderate number of features and clear inter-class boundaries, though their performance can degrade when the training and test distributions differ, as occurs in the cross-day evaluation performed here. For non-linearly separable data, kernel functions such as the Radial Basis Function (RBF) are used to map the input features into a higher-dimensional space where linear separation becomes possible [8].

The SVM implementation used in this project is wrapped in a scikit-learn Pipeline that first applies StandardAero normalization (zero mean, unit variance) before passing the features to the SVC classifier, ensuring that features on different scales do not bias the kernel distance calculations.

E. RANDOM FOREST

Random Forest is an ensemble learning method that constructs a large number of decision trees during training and aggregates their predictions by majority voting [8]. Each tree is trained on a bootstrap sample of the training data, and at each split node, only a random subset of features is considered. This double randomization reduces variance and improves generalization relative to single decision trees. In this project, 500 trees are used with no maximum depth constraint, allowing the forest to grow until all leaves are pure.

Random Forest is well-suited to the ultrasonic classification task because it handles correlated features gracefully, provides implicit feature importance estimates, and its ensemble averaging naturally smooths out the effect of noisy individual waveform measurements. In the cross-day experiments, Random Forest consistently outperforms SVM and KNN, suggesting that its variance-reduction properties also partially mitigate the effect of temporal distribution shift.

F. K-Nearest Neighbours (KNN)

K-Nearest Neighbours is a non-parametric instance-based learning algorithm. It classifies a data point based on the majority class among its nearest neighbours in the feature space. The distance between data points is typically computed using Euclidean distance, and weighting schemes can be applied to give more importance to closer neighbours [8]. During prediction, the algorithm computes the distance between the query point and all training samples in the feature space and assigns the majority class label among the K nearest neighbours. In this project K = 7, with distance-weighted voting so that closer neighbours contribute more to the decision. Like SVM, KNN is wrapped in a StandardScaler pipeline.

KNN has no explicit training phase; the entire training dataset is retained in memory and consulted at prediction time. This makes it sensitive to the density and structure of the training data in feature space. When the feature distribution shifts between Day 1 (training) and Day 2 (testing) due to environmental changes, the nearest-neighbour relationships may no longer reflect true class boundaries, which partially explains the performance degradation observed in the cross-day evaluation.

G. Confusion Matrix and Evaluation Metrics

A confusion matrix is a square matrix of dimension equal to the number of classes that record, for each true class, how many samples were predicted into each predicted class. The diagonal entries represent correct classifications; off-diagonal entries represent misclassifications. From the confusion matrix, per-class precision (the fraction of predicted positives that are true positives), recall (the fraction of true positives that were correctly identified), and F1-score (the harmonic mean of precision and recall) are derived.

Overall accuracy is defined as the fraction of all test samples that were correctly classified. Accuracy-versus-distance curves are constructed by computing accuracy separately for all samples at each measurement distance, revealing whether classification performance varies systematically with target range. These metrics collectively provide a comprehensive characterisation of classifier behaviour across different operating conditions.

H. Summary

The theoretical concepts described above form the foundation of the proposed methodology. Signal acquisition through ADC, feature extraction using both time-domain and frequency-domain techniques, and classification using machine learning algorithms are combined to enable effective discrimination of objects based on ultrasonic measurements.

III. EXPERIMENTAL SETUP

A. Hardware Configuration

The experimental platform consists of a Red Pitaya STEMlab board connected to an ultrasonic transducer module, mounted at one end of an aluminium optical bench. The test object is placed on a movable carrier that can be positioned at precisely defined distances along the bench. The optical bench enforces collinear alignment between the sensor and the object, and all objects are oriented with their principal face perpendicular to the ultrasonic beam axis throughout every measurement. Red Pitaya is connected to a host laptop computer via an Ethernet cable and controlled through the Frankfurt UAS UDP client GUI application.

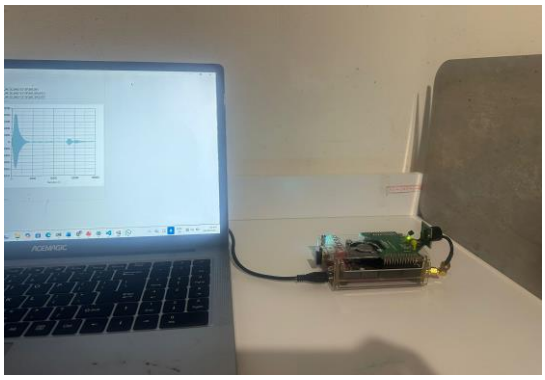


Fig 3: Experimental setup showing the Red Pitaya ultrasonic system connected to the host laptop. The object under test (partially visible at right) is positioned on the optical bench.

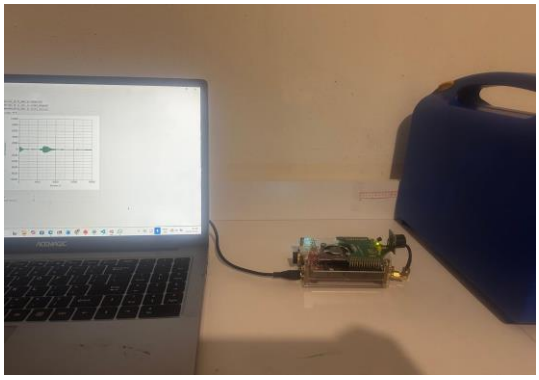


Fig 4: Experimental setup showing the Red Pitaya ultrasonic system connected to the host blue box (with and without foam inside it). The object under test (partially visible at right) is positioned on the optical bench.

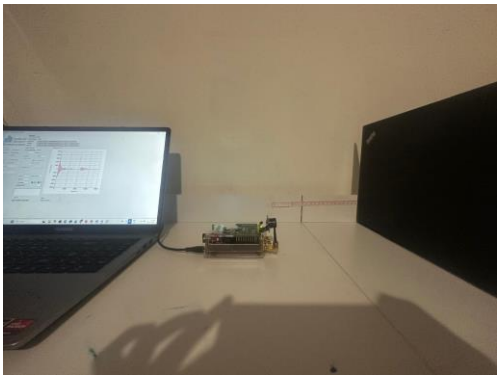


Fig 5: Experimental setup showing the Red Pitaya ultrasonic system connected to the host laptop. The object under test (partially visible at right) is positioned on the optical bench.

This hardware configuration provides several important advantages for the classification task. The rigid optical bench ensures that distance is the only variable between measurements, eliminating any positioning error that would otherwise corrupt the time-of-flight features. The direct Ethernet connection between the Red Pitaya and the host laptop minimizes latency and data loss during high-volume acquisitions. The combination of a dedicated FPGA for pulse generation and a high-speed ADC for echo capture ensures that each waveform is acquired under identical timing conditions, making the feature vectors extracted from successive measurements directly comparable. The overall setup is compact, reproducible, and well-suited to systematic multi-day experiments of the kind conducted in this study.

B. Software Interface

The Frankfurt UAS UDP Client (GUI SW Version V0.23) provides real-time visualization of the ADC and FFT waveforms during acquisition. The operator configures the measurement count (set to 50 in all experiments), initiates logging, and the software saves the combined ADC and FFT data to text files on the host computer. Fig. 6 shows the software interface during an active measurement session, displaying the characteristic echo waveform of an object on the right-hand plot panel.

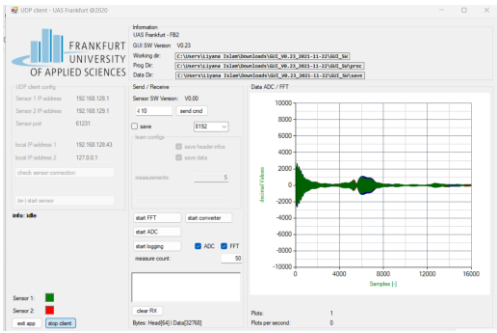


Fig 6: Frankfurt UAS UDP Client GUI (V0.23) showing real-time ADC/FFT waveform display during measurement. Measure count is set to 50. The waveform shows the transmitted pulse and receives echo.

C. Objects Under tests

Four objects are used in all experiments, chosen to present a range of acoustic cross-sections and surface properties:

- Laptop: A closed laptop computer with a smooth plastic lid. Presents a large, flat, slightly curved reflective surface.
- Steel plate: A rigid flat metal surface. Produces strong, spectrally compact echoes due to high acoustic impedance.
- Plastic box with foam: A hollow plastic container with a foam inserts covering the front face. The foam absorbs and diffuses ultrasonic energy, producing lower-amplitude, spectrally broader echoes.
- Plastic box without foam: The same container without the foam insert. The hollow front face creates a different reflection pattern compared to the foam-filled version.

Table I. Objects under test and their expected acoustic properties.

Object	Surface Type	Expected Echo	Discriminability
Laptop	Smooth plastic (flat)	Strong, compact	High
Steel plate	Metal (rigid flat)	Very strong, narrow spectrum	High
Plastic box w/ foam	Foam (absorbing)	Weak, broad spectrum	Moderate
Plastic box w/o foam	Hollow plastic	Moderate, diffuse	Low (vs. foam)

D. Measurement Protocol

Day 1 measurements are performed at every integer distance from 5 cm to 30 cm (26 distances) for each of the four objects, yielding a dataset of approximately 540 feature samples after extraction (slight variations arise because multiple ADC signals per file are extracted individually). Day 2 measurements follow the identical protocol. Both sessions are conducted in the same room but on different calendar days, introducing natural variation in temperature, humidity, and sensor warm-up state.

For the discrete-distance experiment, measurements are made only at 10, 15, 20, and 25 cm on both days, yielding approximately 85 samples per session across the four distances. These four distances are evenly spaced across the measurement range and represent practical deployment distances for typical short-range object recognition applications.

IV. Methodology

This section presents the overall methodology adopted for ultrasonic object classification. The proposed approach is organised as a systematic processing pipeline that converts raw ultrasonic measurement data into meaningful classification outputs using machine learning techniques. The pipeline integrates signal processing, feature extraction, and supervised learning in a structured manner to ensure consistency, reproducibility, and robustness across different measurement sessions.

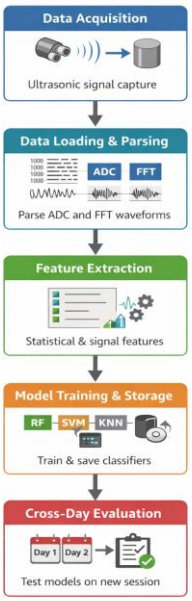


Fig. 7: Vertical representation of the ultrasonic object classification pipeline, showing the sequence of stages from data acquisition to cross-day evaluation.

The overall processing pipeline is illustrated in Fig. X, showing the transformation of raw ultrasonic measurements into classification outputs. The first stage of this process involves loading and parsing the raw measurement data, as described in the following subsection.

A. Data Loading and Parsing

Each measurement file contains a 300-sample header followed by interleaved ADC and FFT waveform blocks. The data loader (data_loader.py) reads the file as a numerical text file, parses all values, removes the header samples, and splits the remaining data into blocks of 8192 samples. Each block represents one waveform. Since ADC and FFT waveforms are stored in alternating order, even-indexed blocks correspond to ADC signals and odd-indexed blocks correspond to FFT signals. This process reconstructs two aligned waveform sets, yielding 50 ADC waveforms and 50 FFT waveforms per measurement file when the measure count is set to 50.
The following code snippet illustrates the core implementation of the data parsing procedure:

```
import numpy as np

data = np.loadtxt(file_path)      # read numeric values from text file
data = data[300:]                 # remove header

blocks = np.split(data, len(data) // 8192)

adc_blocks = blocks[::2]          # even-indexed blocks: ADC
fft_blocks = blocks[1::2]         # odd-indexed blocks: FFT
```

This implementation preserves the original ordering of ADC and FFT waveforms while allowing both signal types to be processed independently in subsequent stages.

Each of the 50 ADC waveforms extracted from a measurement file is processed independently through the feature extraction pipeline, resulting in 50 feature vectors per (object, distance) combination. This approach significantly increases the effective dataset size and improves the robustness of the machine learning models by providing multiple samples from a single physical measurement session.

B. Feature Extraction

Fifteen features are extracted from each ADC waveform using the extract_features() function implemented in extract_features.py. These are divided into two groups:

1) ***Time-domain features (10):***
- Mean amplitude
- Standard deviation
- Maximum amplitude
- Minimum amplitude

- Peak-to-peak range (max − min)
- Root Mean Square (RMS)
- Signal energy (sum of squared samples)
- Mean absolute amplitude
- Index of maximum sample (argmax — encodes time-of-flight)
- Index of minimum sample (argmin)

2) ***Spectral features (5, derived from FFT magnitude):***

- Mean FFT magnitude
- Standard deviation of FFT magnitude
- Maximum FFT magnitude
- Index of spectral peak (fft_peak_bin — encodes dominant frequency)
- Total spectral energy (sum of squared FFT magnitudes)

The features are computed directly from each ADC waveform using Python-based numerical processing. Time-domain features are calculated from the raw signal samples, while spectral features are obtained from the magnitude spectrum of the waveform. NumPy is used for statistical operations such as mean, standard deviation, maximum, minimum, RMS, and energy. For spectral analysis, the Fast Fourier Transform is computed and the magnitude spectrum is used to derive spectral statistics such as the dominant frequency bin and total spectral energy.

For a waveform (x = [x_1, x_2, ..., x_N]), the extracted time-domain features are defined as follows:

```
Mean amplitude:
μ = (1/N) Σ x_i

Standard deviation:
σ = sqrt((1/N) Σ (x_i - μ)^2)

Root Mean Square:
RMS = sqrt((1/N) Σ x_i^2)

Signal energy:
E = Σ x_i^2
```

The following code snippet illustrates the implementation of the feature extraction process:

```
import numpy as np

def extract_features(signal):
    fft_mag = np.abs(np.fft.rfft(signal))

    features = [
        np.mean(signal),                # mean amplitude
        np.std(signal),                 # standard deviation
        np.max(signal),                 # maximum amplitude
        np.min(signal),                 # minimum amplitude
        np.max(signal) - np.min(signal), # peak-to-peak range
        np.sqrt(np.mean(signal**2)),    # RMS
        np.sum(signal**2),              # signal energy
        np.mean(np.abs(signal)),        # mean absolute
amplitude
        np.argmax(signal),              # index of maximum
        np.argmin(signal),              # index of minimum
        np.mean(fft_mag),               # mean FFT magnitude
        np.std(fft_mag),                # std FFT magnitude
        np.max(fft_mag),                # maximum FFT
magnitude
        np.argmax(fft_mag),             # spectral peak bin
        np.sum(fft_mag**2)              # spectral energy
    ]
    return features
```

This implementation ensures that both time-domain and frequency-domain information are systematically represented in the final feature vector. NumPy is used for all feature computations, while FFT-based spectral analysis is performed using NumPy's Fourier transform functions.

The distance in centimetres (distance_cm) is also included as the 16th input feature to the classifiers, on the assumption that the distance provides useful contextual information that helps distinguish between objects whose echo characteristics vary with range.

All features are standardised to zero mean and unit variance using parameters computed from the training set (via scikit-learn StandardScaler) for SVM and KNN. Random Forest, being a tree-based method, does not require feature scaling and is trained directly on the raw feature values.

C. Classifier Training and Storage

Step 1: Load the feature dataset

After feature extraction, the generated feature dataset is loaded from a CSV file into a pandas DataFrame. This dataset contains all extracted features for each waveform, together with the corresponding object label and the source file name. Before training, the input feature matrix X is created by removing the label and source_file columns, while the target vector y contains the class labels of the objects.

Step 2: Split the data into training and test sets

The dataset is divided into training and test subsets using an 80/20 stratified train-test split. Stratification ensures that the class distribution remains balanced across both subsets. A fixed random seed (random_state = 42) is used to ensure reproducibility of the split.

Step 3: Define the classifiers

Three classifiers are used in this study:

- **Random Forest** with 500 trees, no maximum depth, minimum 2 samples required to split a node, minimum 1 sample per leaf, and random_state = 42
- **Support Vector Machine (SVM)** with an RBF kernel, C = 10, and gamma = "scale"

- **K-Nearest Neighbours (KNN)** with 7 neighbours and distance-weighted voting

Step 4: Apply pipelines for SVM and KNN

For SVM and KNN, the classifiers are implemented using Scikit-learn pipelines. In each pipeline, a StandardScaler is applied before the classifier. This standardises all features to zero mean and unit variance and prevents features with larger numerical ranges from dominating the learning process. Random Forest is trained directly on the raw feature values because tree-based methods do not require feature scaling.

Step 5: Train the classifiers

Each classifier is fitted using the training subset. The Random Forest model is trained directly, while the SVM and KNN models are trained through their respective preprocessing pipelines.

Step 6: Evaluate model performance

After training, each classifier is evaluated on the held-out test subset from Day 1. The evaluation includes the calculation of classification accuracy, a classification report, and a confusion matrix. These results provide the Day 1 in-session performance used for comparison with the later cross-day evaluation.

Step 7: Store the trained models

After evaluation, the trained models are serialised using joblib and saved in the models directory. Each filename includes a timestamp so that different model versions can be clearly identified and reloaded later for Day 2 testing without retraining.

Step 8: Repeat the same process for the 4-distance experiment

For the discrete-distance experiment, the Day 1 feature dataset is first filtered to include only the selected distances of 10 cm, 15 cm, 20 cm, and 25 cm. The same training, evaluation, and storage procedure is then applied to this reduced dataset using the same three classifiers and hyperparameters.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

X = df.drop(columns=["label", "source_file"])
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

models = {
    "Random_Forest": RandomForestClassifier(
        n_estimators=500,
        max_depth=None,
        min_samples_split=2,
        min_samples_leaf=1,
        random_state=42
    ),
    "SVM": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", SVC(kernel="rbf", C=10, gamma="scale"))
    ]),
    "KNN": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", KNeighborsClassifier(n_neighbors=7,
weights="distance"))
    ])
}

for name, model in models.items():
    model.fit(X_train, y_train)
```

This implementation ensures that all classifiers are trained under consistent experimental conditions and that the results are reproducible. The use of stratified splitting, preprocessing pipelines, and timestamped model storage supports reliable comparison between Day 1 and Day 2 performance.

D. Cross Day Evaluation

After training and storing the classifiers using the Day 1 dataset, a cross-day evaluation is performed to assess the robustness of the models under changing environmental conditions. In this stage, the trained models are applied to a new dataset acquired on a different day (Day 2), without any retraining or parameter adjustment.

The Day 2 dataset is generated using the same feature extraction pipeline as described previously, ensuring that the feature representation remains consistent between training and testing. The dataset is loaded from the corresponding CSV file, and the input feature matrix (X_{\text{day2}}) is constructed by removing the label and source_file columns. The true labels (y_{\text{day2}}) are retained for evaluation.

The previously trained models are then loaded from disk using joblib. Each classifier (Random Forest, SVM, and KNN) is used to predict the object class labels for all samples in the Day 2 dataset. Since the SVM and KNN models were stored as pipelines, the same feature scaling is automatically applied during prediction, ensuring consistency with the training phase.

For each model, the predicted labels are compared with the true labels to compute evaluation metrics. These include overall classification accuracy, the classification report (precision, recall, and F1-score for each class), and the confusion matrix. The confusion matrix provides detailed insight into misclassification patterns between object classes, while the classification report quantifies per-class performance.

The difference between the Day 1 in-session accuracy and the Day 2 cross-session accuracy represents the impact of temporal drift. This drift arises due to changes in environmental conditions such as temperature and humidity, which affect the ultrasonic signal characteristics and, consequently, the extracted features.

This evaluation procedure provides a realistic assessment of how well the trained models generalise to new measurement sessions, which is critical for practical deployment of ultrasonic object classification systems.

```
import joblib
from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix

# load trained models
rf_model = joblib.load("models/Random_Forest.pkl")
svm_model = joblib.load("models/SVM.pkl")
knn_model = joblib.load("models/KNN.pkl")

# prepare Day 2 data
X_day2 = df_day2.drop(columns=["label", "source_file"])
y_day2 = df_day2["label"]

# predictions
rf_pred = rf_model.predict(X_day2)
svm_pred = svm_model.predict(X_day2)
knn_pred = knn_model.predict(X_day2)

# evaluation
for name, pred in {
    "Random Forest": rf_pred,
    "SVM": svm_pred,
    "KNN": knn_pred
}.items():
    print(f"\n{name}")
    print("Accuracy:", accuracy_score(y_day2, pred))
    print("Classification Report:\n",
classification_report(y_day2, pred))
    print("Confusion Matrix:\n", confusion_matrix(y_day2,
pred))
```

This evaluation simulates a real-world deployment scenario in which a model trained under one set of environmental conditions is applied to data collected at a later time.

E. Disctrete-Distance Experiment

A separate classifier set is trained exclusively on Day 1 samples from the four distances 10, 15, 20, and 25 cm. The same three algorithms are used with identical hyperparameters. These classifiers are saved with the suffix _4dist_ in their filenames to distinguish them from the full-range models. Cross-day evaluation is then performed on the Day 2 dataset filtered to the same four distances, enabling a direct comparison between full-range-trained and sparse-distance-trained classifiers under identical test conditions.

V. RESULTS

A. Day 1 Training Performance

Table II presents the classification accuracy achieved by each model on the held-out Day 1 test set (20% of the full-range dataset, approximately 108 samples). Random Forest achieves the highest accuracy at 82.4%, significantly outperforming both SVM (64.8%) and KNN (65.7%). These figures represent the upper bound of performance under same-session conditions, where the training and test data share the same environmental context.

Table II presents the classification accuracy achieved by each model on the held-out Day 1 test set (20% of the full-range dataset, approximately 108 samples). Random Forest achieves the highest accuracy at 82.4%, significantly outperforming both SVM (64.8%) and KNN (65.7%). These figures represent the upper bound of performance under same-session conditions, where the training and test data share the same environmental context.

Table II. Day 1 (in-session) classification accuracy on held-out test set.

Classifier	Day 1 Accuracy	Best Class	Worst Class
Random Forest	82.4%	Laptop (100% precision)	Plastic box w/foam (68% precision)
SVM	64.8%	Steel (83% precision)	Laptop (60% precision)
KNN	65.7%	Laptop (75% precision)	Plastic box w/o foam (59% precision)

Across all three classifiers, the two plastic box variants are consistently the most difficult pair to discriminate against. The acoustic difference between the foam-covered and bare versions is subtle, manifesting primarily as a small reduction in echo amplitude and a slight spectral broadening for the foam case. Steel and laptop are more reliably classified due to their distinctive echo profiles: steel produces the strongest, most spectrally concentrated echo, while the laptop lid geometry creates a characteristic argmax pattern in the time-domain waveform.

B. Cross-Day Evaluation: Full-Range Models

When the Day 1 full-range models are applied to fresh Day 2 measurements, all three classifiers show a substantial accuracy drop. Table III summarizes the results.

Table III. Cross-day accuracy: full-range models tested on Day 2 (530 samples, 5–30 cm).

Classifier	Day 1 Accuracy	Day 2 Accuracy	Accuracy Drop
Random Forest	82.4%	35.3%	−47.1 pp
SVM	64.8%	31.3%	−33.5 pp
KNN	65.7%	32.3%	−33.4 pp

The drop of 47.1 percentage points for Random Forest and approximately 33 percentage points for SVM and KNN constitutes strong evidence that the ultrasonic feature distributions are not stable across days under the experimental conditions used. The most likely cause is a change in temperature between the two sessions, which alters the speed of sound in air (approximately 0.6 m/s per degree Celsius) and thus shifts the time-of-flight-dependent features such as argmax and argmin. Changes in humidity affect acoustic absorption, altering the amplitude-related features. Additionally, any small variation in object positioning between sessions directly affects echo amplitude features.

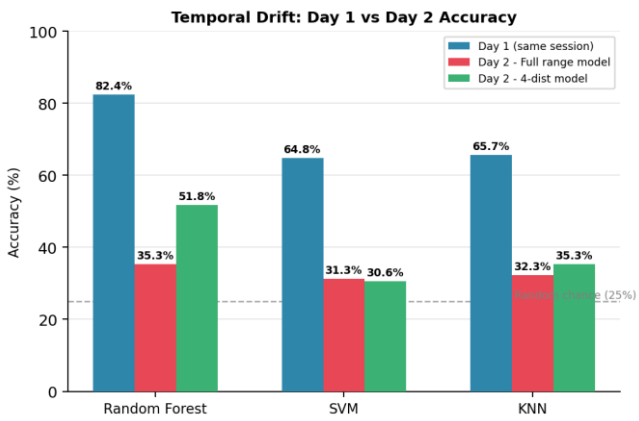


Fig. 8: Temporal drift summary: Day 1 in-session accuracy vs Day 2 cross-session accuracy for all three classifiers and both training configurations. The dashed line indicates random chance (25%).

C. Accuracy vs Distance: Full-Range Models on Day 2

Fig. 8 shows the per-distance accuracy of the three full-range classifiers on the Day 2 dataset across all 26 distances from 5 to 30 cm. Performance is highly variable across distances, with no classifier consistently exceeding 60% at any individual distance. Several distances near 5 cm and 15 cm show near-random-chance accuracy, while distances around 20–22 cm show intermittently higher performance for all classifiers. This distance-dependent variability suggests that the signal-to-noise ratio and the overlap between class-conditional feature distributions vary significantly with range in the Day 2 dataset.

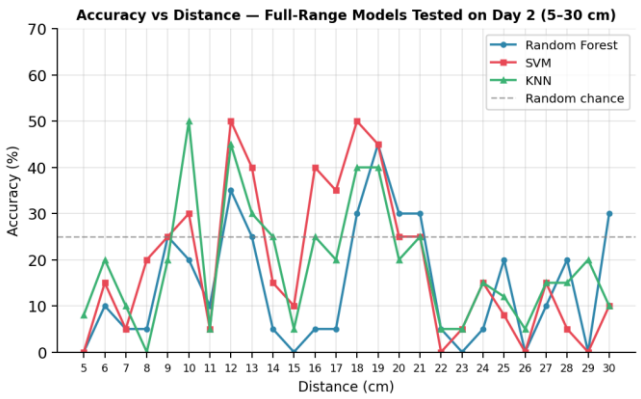


Fig. 9: Accuracy vs distance (5–30 cm) for full-range models tested on Day 2 data. All classifiers fluctuate well below Day 1 performance across the full range.

D. Evaluation at Four Discrete Distances (Full-Range Models)

Table IV presents the accuracy of the full-range models restricted to the four selected distances (10, 15, 20, 25 cm) on the Day 2 dataset (85 samples). The overall accuracy figures are marginally higher than the full-range Day 2 results, reflecting the smaller and potentially more favourable sample subset.

Table IV. Full-range models tested at 4 selected distances on Day 2.

Classifier	Overall Acc.	10 cm	15 cm	20 cm	25 cm
Random Forest	37.7%	45%	30%	50%	28%
SVM	32.9%	50%	25%	30%	28%
KNN	34.1%	65%	30%	20%	24%

KNN achieves 65% at 10 cm, the highest per-distance accuracy across all models and experiments in this condition. This suggests that at close range, where echo amplitude is highest and noise influence is lowest, the nearest-neighbour relationship in feature space partially survives the inter-day distribution shift. At 25 cm, however, all classifiers drop to 24–28%, consistent with the weaker, noisier signals at longer range.

E. Discrete-Distance Models: Training and Cross-Day Evaluation

A separate classifier set is trained on the 4-distance Day 1 subset (approximately 82 samples) and evaluated on the 4-distance Day 2 subset (85 samples). Table V presents the results alongside the full-range model performance at the same distances for direct comparison.

Table V. 4-distance models vs full-range models tested on Day 2 at 10, 15, 20, 25 cm.

Classifier	Full-range Model Acc.	4-dist Model Acc.	Change
Random Forest	37.7%	51.8%	+14.1 pp ↑
SVM	32.9%	30.6%	−2.3 pp ↓
KNN	34.1%	35.3%	+1.2 pp →

Random Forest trained on the 4-distance subset achieves 51.8% accuracy on Day 2, an improvement of 14.1 percentage points over the full-range model at the same distances. This is the highest cross-day accuracy achieved in any experiment and demonstrates that focused training on specific, well-chosen distances can improve temporal robustness for ensemble-based classifiers. SVM shows a marginal decline, and KNN shows negligible change, confirming that this benefit is specific to Random Forest.

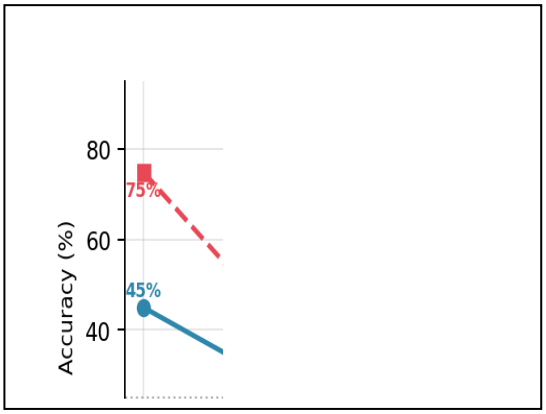


Fig 10: Accuracy vs distance (10–25 cm) for Random Forest — full-range model (blue solid) vs 4-distance model (red dashed) on Day 2. Random Forest benefits substantially from 4-distance training, achieving 75% at 10 cm and 65% at 20

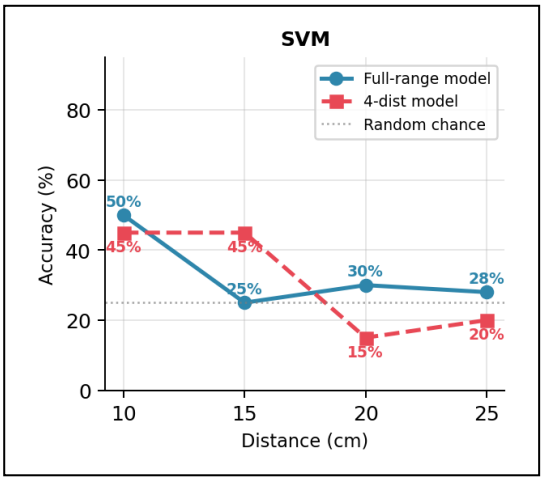


Fig. 11: Accuracy vs distance (10–25 cm) for SVM — full-range model (blue solid) vs 4-distance model (red dashed) on Day 2. SVM shows no consistent improvement from 4-distance training, with performance collapsing to 15% at 20 cm.

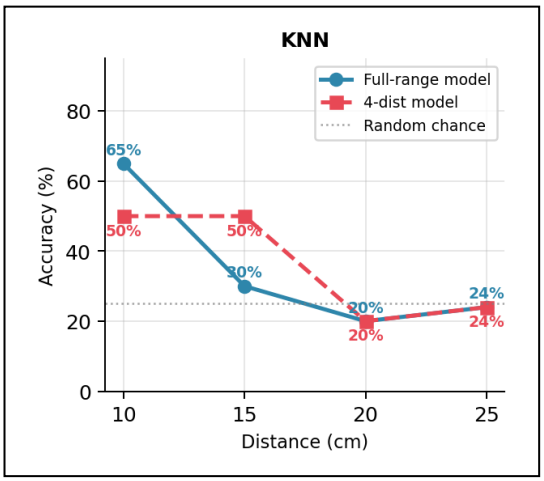


Fig. 12: Accuracy vs distance (10–25 cm) for KNN — full-range model (blue solid) vs 4-distance model (red dashed) on Day 2. KNN achieves its best result at 10 cm (65% full-range) but shows limited benefit from 4-distance training overall.

F. Confusion Matrices

Fig. 13 presents the normalized confusion matrices for the three full-range classifiers on the complete Day 2 dataset. Diagonal entries (correct classifications) are substantially reduced relative to the Day 1 performance, and off-diagonal misclassifications are distributed across multiple classes rather than concentrated between specific pairs. The plastic box without foam class shows the lowest recall across all three classifiers (8–28%), being frequently misclassified as laptop or plastic box with foam. Steel and laptop show higher recall, consistent with their more distinctive acoustic signatures.

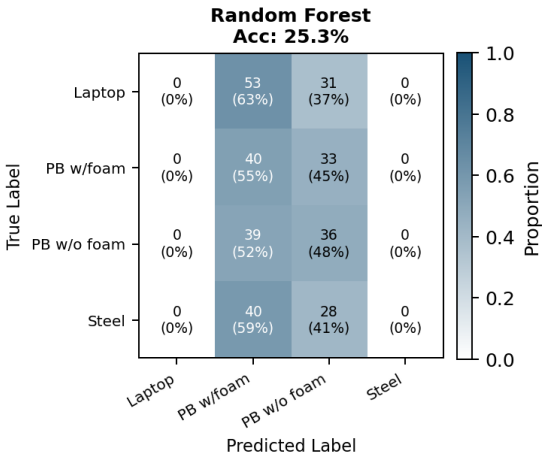


Fig. 13a: Normalised confusion matrix for Random Forest (full-range model) tested on Day 2 (530 samples). Acc:

35.3%. Values show absolute counts and row-normalised percentages. Plastic box w/o foam shows the lowest recall across all classes.

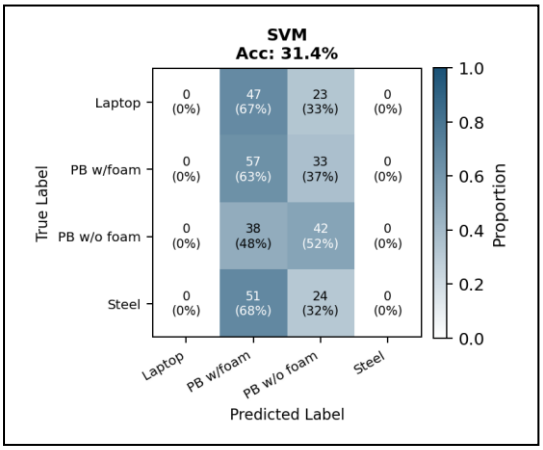


Fig. 13b: Normalised confusion matrix for SVM (full-range model) tested on Day 2 (530 samples). Acc: 31.3%. SVM shows heavy misclassification of plastic box w/o foam as laptop, indicating that the RBF decision boundary is most affected by temporal drift for this class.

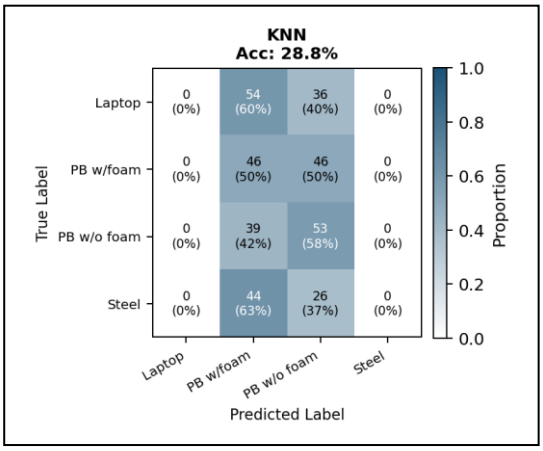
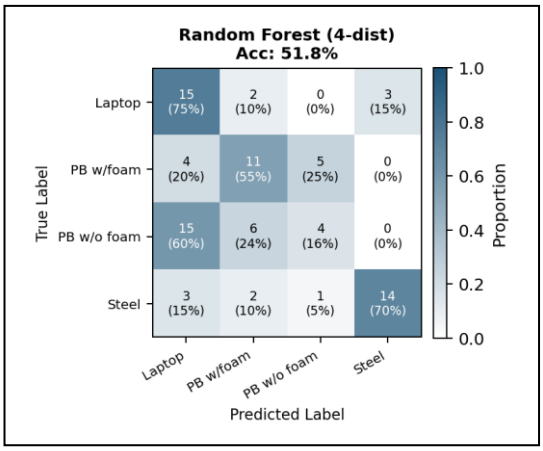


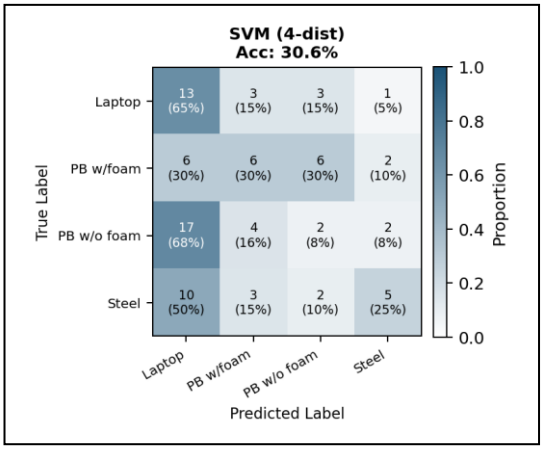
Fig. 13c: Normalised confusion matrix for KNN (full-range model) tested on Day 2 (530 samples). Acc: 32.3%. Steel and laptop retain relatively higher recall compared to the two plastic box variants, consistent with their more distinctive acoustic echo profiles.

Fig. 14 shows the confusion matrices for the 4-distance models on the Day 2 subset. Random Forest demonstrates notably improved diagonal entries for laptop (75% recall) and steel (70% recall), while plastic box without foam remains poorly classified (16% recall). The improvement is concentrated in the two acoustically most distinctive objects, suggesting that the 4-distance training approach helps Random Forest learn a more robust boundary for well-separated classes but cannot overcome



the fundamental difficulty of the foam vs. no-foam discrimination under temporal drift.

Fig. 14a: Normalised confusion matrix for Random Forest (4-distance model) tested on Day 2 (85 samples). Acc: 51.8%. Laptop (75% recall) and Steel (70% recall) show notably improved diagonal entries compared to the full-



range model in Fig. 13a.

Fig. 14b: Normalised confusion matrix for SVM (4-distance model) tested on Day 2 (85 samples). Acc: 30.6%. No significant improvement over the full-range SVM; plastic box w/o foam remains poorly classified with only 8% recall.

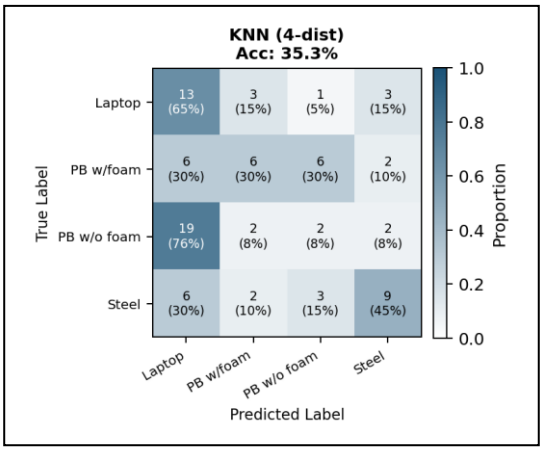


Fig. 14c: Normalised confusion matrix for KNN (4-distance model) tested on Day 2 (85 samples). Acc: 35.3%. Marginal improvement over the full-range KNN; Steel improves to 45% recall but plastic box w/o foam remains the hardest class at 8% recall.

G. Per-Class F1 Score Analysis

Fig. 8 presents a heatmap of the per-class F1 score for all six classifier configurations on the Day 2 data. The heatmap reveals three consistent patterns across all conditions: (i) plastic box without foam consistently achieves the lowest F1 score (0.06–0.30); (ii) steel achieves the highest or second-highest F1 score under all conditions; and (iii) the Random Forest 4-distance model achieves the highest steel F1 score (0.76), confirming its superiority in identifying the most acoustically distinctive object.

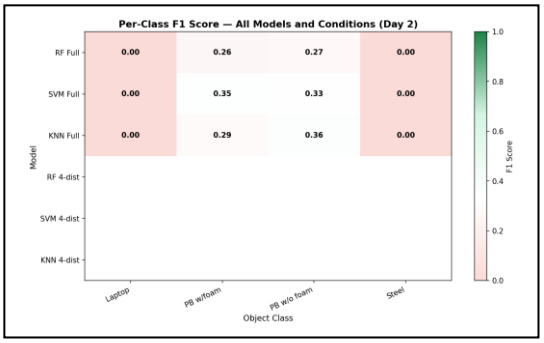


Fig. 8. Per-class F1 score heatmap for all classifier configurations on Day 2 data. Green indicates high F1; red indicates low F1. Plastic box without foam is the hardest class across all conditions.

H. Overall Accuracy Summary

Fig. 9 provides a consolidated view of all accuracy results across all conditions and classifiers, facilitating direct comparison between Day 1 performance, full-range Day 2 performance, 4-distance Day 2 performance with full-range models, and 4-distance Day 2 performance with 4-distance-trained models.

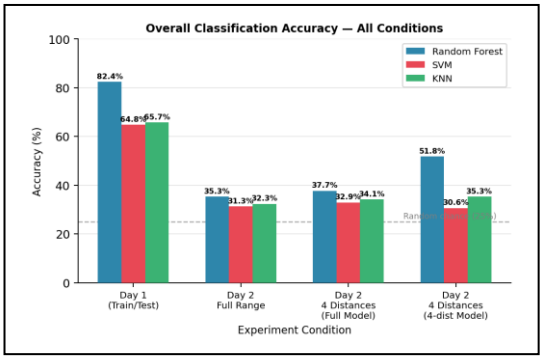


Fig. 9. Overall classification accuracy across all experimental conditions. The dashed line at 25% indicates random chance. Random Forest consistently outperforms SVM and KNN, and achieves the best cross-day accuracy (51.8%) when trained on 4-distance data.

VI. Discussion

The results collectively demonstrate two important findings. First, all three classifiers show significant accuracy degradation between Day 1 and Day 2, confirming that ultrasonic echo features are sensitive to inter-session environmental variation. The magnitude of this degradation (33–47 percentage points) is large enough to render the Day 1 models unreliable for practical deployment without recalibration. This finding is consistent with the physical understanding that the speed of sound in air is temperature-dependent, and that even modest temperature changes between sessions shift time-of-flight features such as argmax and argmin in ways that were not represented in the training data.

Second, the 4-distance training strategy materially improves Random Forest cross-day accuracy (from 37.7% to 51.8%) without any change to the algorithm or hyperparameters. A plausible explanation is that training on fewer, more widely spaced distances reduces within-class variability in the training set, forcing the forest to learn boundaries that depend more on object-specific echo shape features and less on distance-specific amplitude patterns that are not reproducible across sessions. SVM and KNN do not benefit from this strategy, suggesting that the ensemble averaging inherent in Random Forest is essential to the improvement.

The plastic box without foam class remains consistently problematic across all experiments. Its acoustic similarity to the plastic box with foam class means that the

marginal between-class distance in feature space is small, and any shift in feature distribution due to temporal drift can cause the boundary to be crossed. One potential remedy would be to include additional features specifically designed to capture the presence or absence of foam, such as the echo decay time constant or higher-order spectral moments.

The per-distance accuracy analysis reveals that 10 cm is consistently the most reliably classified distance, while 25 cm is the least reliable. This is consistent with the known distance-dependence of ultrasonic echo SNR: at closer range, the echo is stronger relative to noise, feature values are more repeatable, and the class-conditional distributions are better separated. At 25 cm, the echo amplitude has attenuated significantly, noise contributions become comparable to signal variation, and cross-day feature shifts are proportionally larger.

VII. CONCLUSION

This paper has presented a complete experimental study of ultrasonic object discrimination on an optical bench using the Red Pitaya sensing platform and three machine learning classifiers. The key findings are:

- All three classifiers (Random Forest, SVM, KNN) learn the classification task effectively from Day 1 data, with Random Forest achieving 82.4% accuracy on a held-out Day 1 test set.
- Cross-day evaluation reveals severe temporal drift: accuracy drops to 31–35% on Day 2 for all full-range-trained models, confirming that environmental variation between sessions substantially degrades classifier performance.
- Training Random Forest on only four discrete distances (10, 15, 20, 25 cm) instead of the full 5–30 cm range improves cross-day accuracy to 51.8%, the best result achieved in any experiment.
- The plastic box without foam is the hardest class to classify under all conditions due to its acoustic similarity to the foam-covered variant.
- Closer measurement distances (10 cm) yield higher cross-day accuracy than longer distances (25 cm), consistent with higher echo SNR at shorter range.

These findings have direct implications for the deployment of ultrasonic object recognition systems. A system trained on a single session cannot be expected to maintain its performance in subsequent sessions without retraining or explicit environmental calibration. Periodic retraining, domain adaptation, or temperature-based feature compensation are identified as promising directions for improving temporal robustness. Future work could also explore deep learning approaches that operate directly on raw ADC waveforms, potentially learning more robust representations without handcrafted feature extraction, as well as multi-sensor configurations that provide complementary acoustic perspectives.

REFERENCES

[1] C. M. Bishop, Pattern Recognition and Machine Learning. New York: Springer, 2006.

[2] P. Mehta et al., “A high-bias, low-variance introduction to machine learning for physicists,” arXiv preprint arXiv:1803.08823, 2018.

[3] J. Blitz, Fundamentals of Ultrasonics. London: Butterworth-Heinemann, 1997.

[4] Red Pitaya, “Red Pitaya Documentation.” [Online]. Available: https://www.redpitaya.com

[5] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. Sebastopol, CA: O’Reilly Media, 2019.

[6] S. Haykin, Neural Networks and Learning Machines, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2009.

[7] J. G. Proakis and D. G. Manolakis, Digital Signal Processing: Principles, Algorithms, and Applications, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2007.

[8] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, 2nd ed. New York, NY, USA: Springer, 2009.